

软件项目开发与实践

作业 2

基于 Pygame 的 2D RPG 游戏设计与实现 ——以《西游记：祸起观音院》为例

班 级： 软件 2401

学 号： 5120243335

姓 名： 袁杰强

提交日期： 2026 年 6 月 29 日

2026 年 6 月 29 日

目录

1	项目代码仓库与版本控制	1
1.1	公开 Git 仓库链接	1
1.2	提交 (Commit) 历史	1
1.3	版本控制反思	1
2	AI 协作能力专项测评	1
2.1	与 AI 结对编程——调试/功能实现实录	1
2.1.1	问题描述	1
2.1.2	AI 提示词 (Prompt)	2
2.1.3	AI 给出的解决方案	2
2.1.4	最终处理方式	2
3	架构优化	3
3.1	系统概要类图 (优化后)	3
3.2	优化分析	4
3.2.1	优化原因	4
3.2.2	采用的设计模式	4

第 1 章 项目代码仓库与版本控制

1.1 公开 Git 仓库链接

仓库地址: <https://github.com/YJQ-123588/home1>

1.2 提交 (Commit) 历史

序号	提交信息	提交时间
1	实践课-西游记-初始 1.0	2026-06-24 03:29:30
2	Create README.md	2026-06-24 06:22:32
3	Update README.md	2026-06-24 06:24:15
4	西游记 2.0	2026-06-24 06:37:52
5	Merge branch 'main' 合并远端仓库更新	2026-06-24 06:44:41
6	main 分支同时存在几套项目源码	2026-06-24 07:37:38
7	新增 PythonProject3-pygame, 西游记 3.0	2026-06-25 00:12:45
8	仓库同步 1.0/2.0/3.0 三套源码	2026-06-25 01:12:38
9	新增 PythonProject-pygame3.0	2026-06-25 01:45:36
10	新增 PythonProject-pygame3.1	2026-06-25 02:37:57

1.3 版本控制反思

本次项目采用增量式开发策略, 按照功能模块逐步提交:

- (1) **初始 1.0 版本:** 完成基础游戏框架, 包含玩家移动、场景切换等核心功能;
- (2) **2.0 版本:** 新增战斗系统、NPC 对话、存档功能;
- (3) **3.0 版本:** 完善战斗逻辑、增加避战选项、优化地图碰撞;
- (4) **3.1 版本:** 修复 HP 上限 bug、完善存档系统、增加状态栏 UI。

每次提交都对应一个完整的功能模块, 便于追溯和回滚。遇到 Bug 时单独提交修复, 确保版本稳定性。

第 2 章 AI 协作能力专项测评

2.1 与 AI 结对编程——调试/功能实现实录

2.1.1 问题描述

在开发战斗系统时, 遇到了战斗状态机逻辑混乱的问题。具体表现为:

- (1) 玩家攻击后, 怪物反击动画不播放;
- (2) 战斗结束条件判断错误, 导致无限循环;
- (3) 大招解锁状态在场景切换后丢失。

2.1.2 AI 提示词 (Prompt)

“我在开发一个 *Pygame RPG* 游戏的战斗系统, 使用有限状态机管理战斗流程。当前问题是:

- (1) *STATE_PLAYER_ATK* 状态执行完后, 没有正确切换到 *STATE_MONSTER_HURT*
- (2) 战斗结束判断条件有问题, 击败怪物后无法正常退出战斗
- (3) *ultimate_unlocked* 变量在场景切换后被重置

请帮我分析 *fight_scene.py* 中的状态机逻辑, 找出问题所在并提供修复方案。
”

2.1.3 AI 给出的解决方案

- (1) 检查状态转换条件, 确保每个状态都有明确的 *next_state*;
- (2) 添加战斗结束标志, 在怪物 $HP \leq 0$ 时正确设置 *fight_over=True*;
- (3) 将 *ultimate_unlocked* 状态同步到 *Game* 主控类, 实现场景间持久化。

2.1.4 最终处理方式

选择 **B**: 理解思路后, 手动重写了代码并进行了适配。

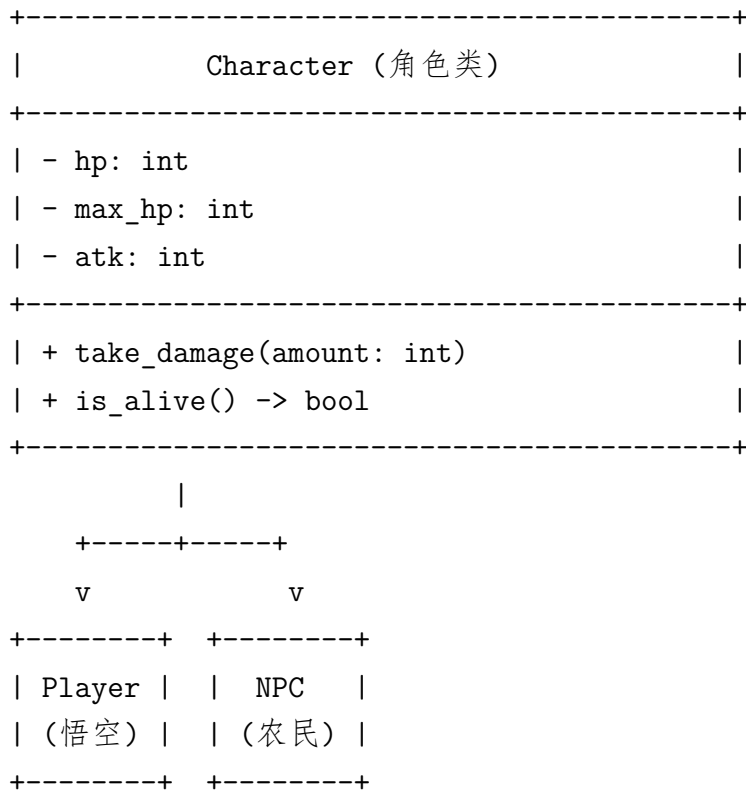
- 重新设计了状态机的状态转换表;
- 添加了状态持续时间控制, 避免状态跳转过快;
- 将关键状态变量同步到 *Game* 类实现持久化。

学到的最重要一课: AI 能够快速定位问题并提供解决思路, 但需要开发者理解底层逻辑后进行适配, 而不是盲目复制代码。

第 3 章 架构优化

3.1 系统概要类图（优化后）





3.2 优化分析

3.2.1 优化原因

- (1) **单一职责原则**: Game 类只负责状态管理, 场景类只负责渲染和交互;
- (2) **组合模式**: 场景包含角色, 角色包含动画;
- (3) **场景模板方法**: 基类定义流程, 子类实现具体逻辑;
- (4) **状态持久化**: 关键数据同步到 Game 类, 实现跨场景保存。

3.2.2 采用的设计模式

- (1) **单例模式**: Game 类作为全局唯一状态管理器;
- (2) **观察者模式**: 状态变化时通知相关组件更新;
- (3) **状态模式**: 战斗系统使用有限状态机管理流程;
- (4) **模板方法模式**: 场景基类定义渲染流程, 子类重写具体实现。